

Mathematical Formulations, Scoring Functions, and Evaluation Metrics

News Recommendation & Retrieval Engine (MIND & EB-NeRD)

Kavan Gandhi · CS4.406 Information Retrieval & Extraction

August 2026 · Codebase: IRE-Assignment-1

This document details all mathematical formulas, scoring functions, objective formulations, evaluation metrics, and statistical methods implemented across the project codebase.

1 Lexical Candidate Generation and Retrieval: Okapi BM25

Implemented in `src/bm25.py`, `src/generate_mind_submission.py`, and `src/generate_ebnerd_submission.py`.

1.1 Robertson–Spärck Jones Inverse Document Frequency (IDF)

$$\text{IDF}(t) = \ln \left(1 + \frac{N - \text{df}(t) + 0.5}{\text{df}(t) + 0.5} \right) \quad (1)$$

- N : Total number of articles in the indexed catalog corpus.
- $\text{df}(t)$: Document frequency of term t (count of documents in the catalog containing t).
- $+0.5$: Robertson–Spärck Jones smoothing parameter preventing division-by-zero and negative term weights for high-frequency terms.
- **Significance**: Quantifies the specificity and discriminative capacity of a query term. Common non-discriminative terms receive near-zero weights, while rare, topical terms receive high positive weights.

1.2 Okapi BM25 Document Scoring Function

$$\text{Score}_{\text{BM25}}(Q, D) = \sum_{t \in Q \cap D} \text{IDF}(t) \cdot \frac{\text{TF}(t, D) \cdot (k_1 + 1)}{\text{TF}(t, D) + k_1 \cdot \left(1 - b + b \cdot \frac{|D|}{\text{avgl}} \right)} \quad (2)$$

- $\text{TF}(t, D)$: Term frequency of token t in candidate document D .
- $|D|$: Document length (total token count of article title + abstract).
- avgl : Average document length across all N catalog articles:

$$\text{avgl} = \frac{1}{N} \sum_{i=1}^N |D_i| \quad (3)$$

- $k_1 = 1.5$: Term frequency saturation parameter. As $\text{TF}(t, D) \rightarrow \infty$, the contribution of term t asymptotes toward $(k_1 + 1)$.
- $b = 0.75$: Document length penalization parameter ($b = 0$ corresponds to no length penalty; $b = 1$ scales inversely with relative length).
- **Significance**: Computes the lexical relevance between pseudo-query Q (concatenated recent user click history) and candidate article D , balancing term frequency saturation against document verbosity.

2 Dense Semantic Candidate Retrieval and User Representation

Implemented in `src/embeddings.py` and `src/eval_embeddings.py`.

2.1 Vector L_2 Normalization

$$\hat{\mathbf{v}} = \frac{\mathbf{v}}{\|\mathbf{v}\|_2 + \epsilon} = \frac{\mathbf{v}}{\sqrt{\sum_{k=1}^d v_k^2 + \epsilon}} \quad (4)$$

- $\mathbf{v} \in \mathbb{R}^d$: Dense text embedding vector ($d = 384$ for all-MiniLM-L6-v2 / Multilingual BERT).
- $\epsilon = 10^{-12}$: Small numerical stability constant.
- **Significance**: Maps vectors to the unit hypersphere $\mathbb{S}^{d-1}$ so that inner products correspond to exact cosine similarities.

2.2 User History Centroid Representation (Mean Pooling)

$$\mathbf{u} = \text{normalize} \left(\frac{\sum_{i=1}^m w_i \cdot \mathbf{e}_{a_i}}{\sum_{i=1}^m w_i + \epsilon} \right) \quad (5)$$

- $H_u = [a_1, a_2, \dots, a_m]$: Chronologically ordered list of up to $m = 20$ most recent clicked article IDs prior to the split timestamp cutoff.
- $\mathbf{e}_{a_i} \in \mathbb{R}^d$: L_2 -normalized dense embedding of historical article a_i .
- w_i : Historical click weight (uniform $w_i = 1.0$ or exponential decay weight).
- **Significance**: Synthesizes the user's sequential reading history into a single compact semantic centroid vector without future data leakage.

2.3 Semantic Similarity Scoring

$$\text{Score}_{\text{Dense}}(u, D) = \langle \hat{\mathbf{u}}, \hat{\mathbf{e}}_D \rangle = \sum_{k=1}^d \hat{u}_k \cdot \hat{e}_{D,k} \quad (6)$$

- **Significance**: Measures the continuous cosine similarity between user vector $\hat{\mathbf{u}}$ and candidate article vector $\hat{\mathbf{e}}_D$. Exact or approximate K -nearest neighbors are retrieved using FAISS (IndexFlatIP or IndexHNSWFlat).

3 Feature Engineering: Temporal Recency Decay

Implemented in `src/feature_store.py`.

$$\text{Recency Score}(u) = \sum_{i \in H_u} 0.5^{\frac{\text{age}(i)}{h}} \quad (7)$$

- $\text{age}(i)$: Age in hours of click i measured relative to the split earliest cutoff timestamp $t_{\text{as.of}}$:

$$\text{age}(i) = \frac{t_{\text{as.of}} - t_{\text{click},i}}{3600} \quad (8)$$

- $h = 24.0$ hours: Exponential decay half-life.
- **Significance**: Summarizes user freshness into a scalar feature. Recent interactions contribute heavily (a click 24 hours ago contributes 0.5; 48 hours ago contributes 0.25), distinguishing active users from inactive ones.

4 Ranking and Accuracy Evaluation Metrics

Implemented in `src/metrics.py` and `src/eval_harness.py`.

4.1 Area Under the ROC Curve (AUC — Wilcoxon–Mann–Whitney)

$$\text{AUC} = \frac{\sum_{i \in \text{Pos}} \text{rank}(i) - \frac{n_{\text{pos}}(n_{\text{pos}}+1)}{2}}{n_{\text{pos}} \cdot n_{\text{neg}}} \quad (9)$$

- $\text{rank}(i)$: 1-based rank position of the i -th positive article when candidates are sorted in **ascending** order of predicted scores (with average rank tie adjustments).
- $n_{\text{pos}}, n_{\text{neg}}$: Number of clicked ($y = 1$) and unclicked ($y = 0$) candidate articles in the impression.
- **Significance**: Measures pairwise classification quality, representing the exact probability that a randomly selected clicked article is scored higher than an unclicked article:

$$\text{AUC} = P(s_{\text{pos}} > s_{\text{neg}}) \quad (10)$$

4.2 Mean Reciprocal Rank (MRR)

$$\text{RR} = \begin{cases} \frac{1}{\text{rank}_{\text{first_pos}}}, & \text{if } n_{\text{pos}} \geq 1 \\ 0.0, & \text{if } n_{\text{pos}} = 0 \end{cases}, \quad \text{MRR} = \frac{1}{|U|} \sum_{u=1}^{|U|} \text{RR}_u \quad (11)$$

- $\text{rank}_{\text{first_pos}}$: 1-based rank of the top-ranked clicked article when candidate items are sorted in **descending** order of scores.
- **Significance**: Evaluates how quickly the user encounters their first relevant article.

4.3 Normalized Discounted Cumulative Gain (nDCG@K)

$$\text{DCG@K} = \sum_{i=1}^K \frac{y_i}{\log_2(i+1)}, \quad \text{IDCG@K} = \sum_{i=1}^{\min(K, n_{\text{pos}})} \frac{1}{\log_2(i+1)} \quad (12)$$

$$\text{nDCG@}K = \frac{\text{DCG@}K}{\text{IDCG@}K} \quad (13)$$

- $y_i \in \{0, 1\}$: Binary click ground-truth label at rank position i .
- $\frac{1}{\log_2(i+1)}$: Position discount factor dampening gains at lower ranks.
- $\text{IDCG@}K$: Ideal Discounted Cumulative Gain achieved by placing all positive articles at the top.
- **Significance**: Position-sensitive ranking metric that heavily rewards placing relevant news at the topmost slots ($K \in \{5, 10\}$), normalized to $[0, 1]$.

4.4 Candidate Retrieval Recall@K

$$\text{Recall@}K = \frac{|\text{Retrieved@}_K \cap \text{GroundTruth}|}{|\text{GroundTruth}|} \quad (14)$$

- Retrieved@_K : Set of top- K articles retrieved from the full catalog ($K \in \{50, 100, 200\}$).
- GroundTruth : Set of true clicked articles in the impression.
- **Significance**: Measures the retrieval recall ceiling of the candidate generation stage prior to fine-grained reranking.

5 Beyond-Accuracy Recommendation Metrics

Implemented in `src/metrics.py`.

5.1 Intra-List Diversity (ILD@K)

$$\text{ILD@}K = \frac{2}{K(K-1)} \sum_{1 \leq i < j \leq K} (1 - \langle \hat{\mathbf{e}}_i, \hat{\mathbf{e}}_j \rangle) \quad (15)$$

- $\hat{\mathbf{e}}_i, \hat{\mathbf{e}}_j$: Normalized embedding vectors of recommendations at ranks i and j .
- $1 - \langle \hat{\mathbf{e}}_i, \hat{\mathbf{e}}_j \rangle$: Pairwise cosine distance.
- **Significance**: Measures the semantic spread among recommended articles in a user's top- K slate ($K = 10$), penalizing narrow topic redundancy.

5.2 Novelty@K (Self-Information Surprise)

$$\text{Novelty@}K = \frac{1}{K} \sum_{i=1}^K -\log_2 P(a_i) \quad (16)$$

- $P(a_i)$: Prior empirical click probability (popularity) of article a_i in historical logs:

$$P(a_i) = \frac{\text{Clicks}(a_i)}{\sum_{j \in C} \text{Clicks}(a_j)} \quad (17)$$

- **Significance**: Quantifies the average information content (surprise in bits) of the recommendations. Recommending viral head articles yields low novelty, whereas recommending niche tail articles yields high novelty.

5.3 Catalog Coverage@K

$$\text{Coverage@}K = \frac{|\bigcup_{u \in U} \text{TopK}_u|}{|C|} \quad (18)$$

- TopK_u : Set of top- K recommended articles presented to user u .
- $|C|$: Total number of articles in the full catalog.
- **Significance**: Proportion of the catalog exposed to at least one user, assessing catalog discovery and long-tail exploration.

6 Statistical Inference: Non-Parametric Bootstrap Confidence Intervals

Implemented in `src/metrics.py`.

Given empirical metric observations $S = [s_1, s_2, \dots, s_N]$ over N test impressions:

1. Draw $B = 1000$ bootstrap resamples S^{*b} of size N with replacement from S .

2. Compute the replicate mean for each resample:

$$\bar{\theta}^{*b} = \frac{1}{N} \sum_{j=1}^N s_j^{*b}, \quad \forall b \in \{1, 2, \dots, B\} \quad (19)$$

3. Calculate the empirical 95% Confidence Interval via percentiles:

$$CI_{95\%} = [\text{Percentile}(\bar{\theta}^*, 2.5\%), \text{Percentile}(\bar{\theta}^*, 97.5\%)] \quad (20)$$

- **Significance:** Yields non-parametric, statistically rigorous confidence intervals for all ranking and retrieval metrics without assuming Gaussian error distributions.